

3B Window Functions

Benjamin Bravo

2026-02-13

Planned Approach

First, I will find a dataset that includes daily time series data for at least two items, such as stock prices or cryptocurrency prices starting from January 1, 2022. The dataset will include the item name, date, and closing price.

Next, I will clean and organize the data. This includes making sure the dates are formatted correctly and that the data is sorted in order from earliest to latest. Window functions only work properly when the data is ordered correctly.

Then, I will calculate the year-to-date (YTD) average for each item. This means I will group the data by item and year and compute the running average from the start of each year to each date.

After that, I will calculate the six-day moving average for each item. This will take the current day and the previous five days to compute a rolling average. This helps smooth out short-term changes in the data.

Finally, I will review the results and compare the original values to the calculated averages.

Anticipated Data Challenges

One challenge may be missing dates, such as weekends or holidays in stock data. While this does not stop the calculations, it can affect how the moving average looks.

Another challenge is making sure the data is correctly grouped and ordered. If the data is not sorted by date, the calculations may be incorrect.

If the dataset covers multiple years, I will need to make sure the year-to-date average resets at the beginning of each year.

Introduction

This analysis looks at the daily stock prices of Apple (AAPL) from January 3, 2022 to January 10, 2022 obtained from using Yahoo Finance in R. Using time series window functions, we calculate the Year-to-Date (YTD) average and the 6-day moving average to better understand short-term trends. These measures help smooth daily price changes and make it easier to see the overall direction of the stock during this period.

Using Yahoo Finance in R

```
install.packages("tidyquant")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'  
## (as 'lib' is unspecified)
```

```
library(tidyquant)
```

```
## Registered S3 method overwritten by 'quantmod':  
##   method              from
```

```
## as.zoo.data.frame zoo

## -- Attaching core tidyquant packages ----- tidyquant 1.0.11 --
## v PerformanceAnalytics 2.0.8      v TTR      0.24.4
## v quantmod      0.4.28      v xts      0.14.1
## -- Conflicts ----- tidyquant_conflicts() --
## x zoo::as.Date()      masks base::as.Date()
## x zoo::as.Date.numeric()      masks base::as.Date.numeric()
## x PerformanceAnalytics::legend() masks graphics::legend()
## x quantmod::summary()      masks base::summary()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

stocks <- tq_get(c("AAPL", "MSFT", "TSLA"),
  from = "2022-01-01",
  to = Sys.Date())

head(stocks)

## # A tibble: 6 x 8
##   symbol date      open  high   low close  volume adjusted
##   <chr> <date>    <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
## 1 AAPL  2022-01-03  178.  183.  178.  182.  104487900  178.
## 2 AAPL  2022-01-04  183.  183.  179.  180.   99310400  176.
## 3 AAPL  2022-01-05  180.  180.  175.  175.   94537600  171.
## 4 AAPL  2022-01-06  173.  175.  172.  172.   96904000  168.
## 5 AAPL  2022-01-07  173.  174.  171.  172.   86709100  168.
## 6 AAPL  2022-01-10  169.  172.  168.  172.  106765600  168.

library(dplyr)

##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
## #
## #####
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:xts':
##
##   first, last
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
```

```
##
## intersect, setdiff, setequal, union

stocks_calculated <- stocks %>%
  arrange(symbol, date) %>%
  group_by(symbol, year = format(date, "%Y")) %>%
  mutate(
    YTD_Avg = cummean(adjusted),          # Year-to-date average
    MA_6 = rollmean(adjusted, 6, fill = NA, align = "right") # 6-day moving average
  ) %>%
  ungroup()

head(stocks_calculated)
```

```
## # A tibble: 6 x 11
##   symbol date      open  high   low close  volume adjusted year  YTD_Avg MA_6
##   <chr> <date>      <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl> <chr>   <dbl> <dbl>
## 1 AAPL  2022-01-03  178.  183.  178.  182.  1.04e8    178.  2022    178.  NA
## 2 AAPL  2022-01-04  183.  183.  179.  180.  9.93e7    176.  2022    177.  NA
## 3 AAPL  2022-01-05  180.  180.  175.  175.  9.45e7    171.  2022    175.  NA
## 4 AAPL  2022-01-06  173.  175.  172.  172.  9.69e7    168.  2022    173.  NA
## 5 AAPL  2022-01-07  173.  174.  171.  172.  8.67e7    168.  2022    172.  NA
## 6 AAPL  2022-01-10  169.  172.  168.  172.  1.07e8    168.  2022    172.  172.
```

Conclusion

Between January 3, 2022 and January 10, 2022, Apple's Year-to-Date (YTD) average decreased from 178.10 to 171.73. This means that the stock price generally moved downward during that first week of the year.

Because the YTD average is a running average starting from January 1, each new day's price affects the overall average. If the daily closing prices are lower than earlier days, the YTD average will decrease. That is what happened during this period — the stock experienced short-term declines, which pulled the average down.

This suggests that Apple had a negative short-term trend during the first week of January 2022.